

Python: Unleash your Inner Coder

A SAS Programmer's Guide to Python

Minnesota SAS Users Group

22 July 2026

Charu Shankar
SAS Education



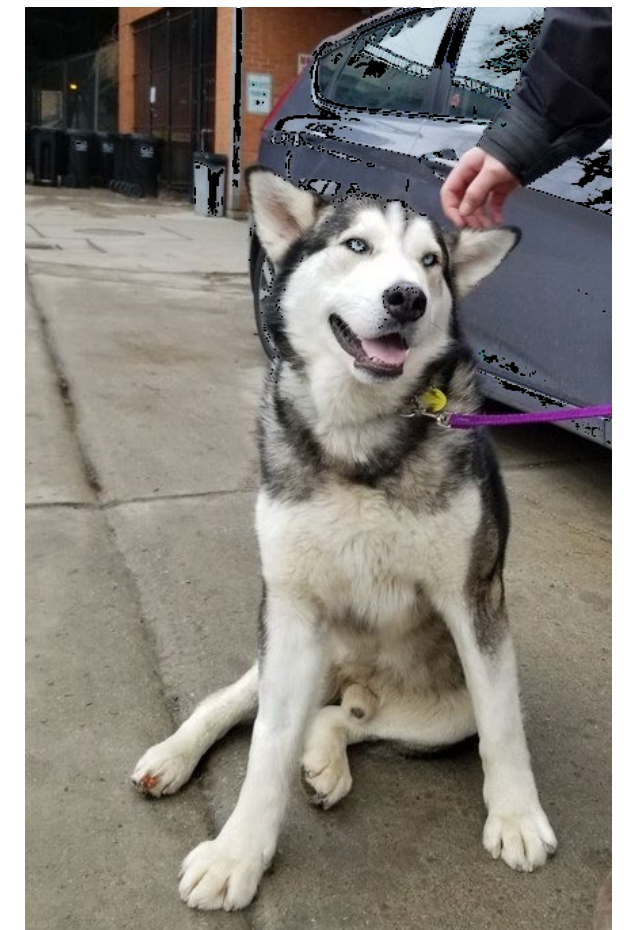
Bio

With a background in computer systems management. SAS Instructor Charu Shankar engages with logic, visuals, and analogies to spark critical thinking.

A SAS Instructor since 2007, she curates and delivers unique content via the SAS YouTube channel, SAS global forum, SAS Ask the Expert Series, SAS Training Post Blog, etc.

Charu loves to support users by teaching at conferences on topics related to SAS, SQL, Efficiencies, PERL, Macros, Python, Viya, etc.

When she's not coding, Charu is A Yoga Instructor who loves to explore Canadian trails with her husky Miko.



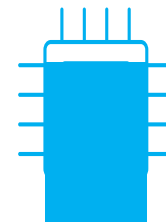
Agenda



Introduction



Python & A.I.



Basic Python



Python Data Manipulation



Handy Links

1 Introduction

What Is Python

Python application in analytics industry

Why should you learn Python to work with SAS

What Is Python?

Python is a powerful, yet simple language. It's versatile, easy to read, and is used for everything from data analysis to web development.

With Python, you can quickly solve problems and automate tasks with clear, well-planned code.



Python application in analytics industry

- Python supports **object-oriented programming** structured programming, and functional programming patterns.
- Python is often called ‘**the second-best language** for everything.’ As a jack of all trades, Python is widely used across analytics—from data manipulation to machine learning—and organizations see real advantage in standardizing on it. Its ecosystem now exceeds 500,000 packages on PyPI and grows daily.
- Python’s inherent readability and simplicity with the vast number of **libraries** available helps data scientists find packages tuned to their needs.

Why Should You Learn Python to Work with SAS?



Learning Python alongside SAS is like mastering two complementary tools—each has its strengths. Python’s libraries, like pandas and NumPy, extend SAS’s capabilities, giving you more control over your data, while allowing you to automate tasks and perform complex analyses.

Together, they make for a powerful combination for tackling complex data challenges.

2 Python & AI: What You Still Need to Know

- AI writes code
- AI helps explain and debug code
- You provide understanding and judgment

Python & AI: What You Still Need to Know

- AI coding assistants like SAS Viya Copilot and ChatGPT act as a modern **pair programmer**—suggesting Python syntax, explaining errors, and drafting functions as you type.
- **speed up learning** ask for a plain-English explanation of a loop, generate example code, or translate a SAS DATA step into equivalent Python.
- **Treat AI as a helper**, not an authority — always review, test, and understand the code before putting it into production.

SAS Data & AI Studio (create.demo.sas.com/SASDataAIStudio) is a cloud-based environment where you can learn SAS, PROC PYTHON, pandas, and AI-assisted coding in one place by writing, running, debugging, and validating code against real data workflows.



3 Python Basics for SAS Programmers

- Python vs SAS
- Python Programming Language Basics
- Functions
- Data structures

All code examples in this session are tested on Python 3.14.

Python vs. SAS basics

	Python	SAS Data step
Case Sensitivity	Python respects casing of variable names or functions. E.g, Python will view City and city as two completely different variables, despite them having the same spelling because the casing of each letter is inconsistent.	SAS Data step will treat both City and city as the same variable. Variable names are case insensitive
Order of first occurrence	Python generally processes the code from top to bottom, so if it encounters a variable not yet defined, it would throw a NameError.	SAS also processes code from top to bottom. During execution if it encounters a variable not yet defined, it assigns a missing value to that variable
Slicing	Slicing in Python allows you to extract a subset of a sequence (like a list, tuple, or DataFrame) using indices and range notation	Slicing in SAS typically involves using PROC SQL or data step filtering techniques to select specific rows and columns from a dataset based on conditions or positions.
Storage	Data Frame	Data Set/Table
End of program statement	No symbols are used to end a program statement. The end-of-line character is used to end a Python statement	semicolon

Python Programming Basics – Comments

Documentation is an important step in programming. Comments tell a computer to ignore that part of the program during execution and helps users provide content on why something is written the way it is.

A comment in Python starts with the # (pound symbol/hash sign/hash tag) and a space:

```
# don't run this bit - it's a comment  
print('Today is Tuesday')
```

```
Today is Tuesday
```



The shortcut for adding (or removing) comment lines in Python editors is the same as in SAS Studio: CTRL + /

Python Programming Basics – Indenting

Indentation in Python is used to identify lines of code executed together. The block of code that is part of **for** loop has to be indented so that Python knows what to execute for each iteration of the loop.

This code will help convert Fahrenheit temperatures to Celsius for visiting Canadians.

```
[1]: #3.1 indenting
chictemp = [59, 60, 61, 62, 63]
for i in chictemp:
    celsius = (i - 32) * 5/9
    print('Temp for a canadian is:', celsius, 'celsius')
```

```
Temp for a canadian is: 15.0 celsius
Temp for a canadian is: 15.555555555555555 celsius
Temp for a canadian is: 16.111111111111111 celsius
Temp for a canadian is: 16.666666666666668 celsius
Temp for a canadian is: 17.222222222222222 celsius
```

Indenting Matters

The following code snippet is identical to the snippet in the previous slide. Except.. The line after the **for** block is **not** indented here. This results in a error being raised in the interpreter.

```
[2]: #3.1 indenting
chictemp = [59, 60, 61, 62, 63]
for i in chictemp:
celsius = (i - 32) * 5/9
print('Temp for a canadian is:', celsius, 'celsius')
```

```
Cell In[2], line 4
    celsius = (i - 32) * 5/9
    ^
```

IndentationError: expected an indented block after 'for' statement on line 3

Python Basics – ending program statement

No symbols are used to end a program statement. The end-of-line character is used to end a Python statement. This also helps to enforce legibility by keeping each statement on a separate physical line.

Coincidentally, like SAS, Python will also accept a semi-colon as an end of statement terminator. This is rarely seen because multiple statements on the same physical line is considered to be unacceptable.

Inconsistent Casing

```
In[3]: #3.1 Case sensitivity  
city = 'Chicago'  
print(City)  
# This will output a NameError because  
# the casing is not consistent.
```

NameError

Traceback (most recent call last)

Cell In[3], line 3

```
1 #3.1 Case sensitivity  
2 city = 'Chicago'  
----> 3 print(City)  
4 # This will output a NameError because  
5 # the casing is not consistent.
```

NameError: name 'City' is not defined

Order of First Occurrence

```
[4]: #3.1 order of first occurrence  
print(year)  
year=2026
```

NameError

Traceback (most recent call last)

Cell In[4], line 2

```
1 #3.1 order of first occurrence  
----> 2 print(year)  
3 year=2026
```

NameError: name 'year' is not defined

Order of first occurrence - SAS

```
6      /*3.1 order of first occurrence*/
7      data _null_;
8          x=year;
9          year=2026;
10         put x=;
11     run;

X=.
```

Functions: Reusable Blocks of Code

- Python has several built-in functions.
- Function names are generally typed in lowercase.
- Defining your own functions -an important part of python programming

Some of Python's built-in functions include the following:

Function	Description
print()	Prints to the screen
type()	Returns the type of an object
sorted()	Returns a sorted list ¹

```
#3.2 built in functions  
print('Hello')
```

```
a = 'Test'  
print(type(a))
```

```
a = [5,1,0,3,2,4]  
print(sorted(a))
```

```
Hello  
<class 'str'>  
[0, 1, 2, 3, 4, 5]
```

Python Data Structures: Organizing Your Data

- Data structures are containers for data and other objects.
- They form building blocks and aid the creation of Dataframes and other useful objects.
- DataFrames are built on top of Python and Numpy modules.
- Python data structures can be broken down into:
 - Basic
 - Advanced

Python Basics – Variable Names

A Variable is just a way to store data, objects, lists etc.

- Variable naming rules – names can contain letters, numbers, and underscores¹, and must begin with a letter or an underscore.
- Case matters: the variables 'a' and 'A' are not the same.
- Python supports Unicode variable names.
- Python has some reserved words (again, case matters):

Keyword	Why Python reserves it
if	conditional logic
else	alternate branch
while	loop control
import	load a package
class	object-oriented programming
True / False	Boolean values
None	missing/null object

```
[6]: #3.4 Variable names
a = 12
A = 'Twelve'
print(a,A)

υψος = 10
print(υψος)

12 Twelve
10
```


Python Data Structures – Basic Data Types

The Building Blocks of Your Code

There are several basic data types in Python, including the following¹:

Data Type
Str
Int
Float
Bool

¹Other data types include list, tuple, dict.

Python Data Structures – String

A String can be defined as the sequence of characters within quotation marks. In python, single, double, or triple quotes can be used to define a string. Consider the string examples below:

```
[7]: #3.4 The operator + is used to concatenate 2 strings as the operation returns "hello world"  
     'hello' + ' world'
```

```
[7]: 'hello world'
```

```
[8]: #3.4 The operator * is known as the repetition operator as the operation returns "hello hello"  
     'hello ' * 2
```

```
[8]: 'hello hello '
```

Python Data Structures - string in SAS

Here is how we would concatenate a string in SAS

```
19  /* 3.4 SAS string operation*/
20  data _null_;
21      x='hello';
22      y=' world';
23      z=catt(x,y);
24      put z;
25  run;
```

hello world

Python Data Structures - String Slicing & Indexing

- Python provides indexing methods for a number of objects, including lists. SAS has a similar construct `_n_` for the Data Step. Both indexing methods act as keys providing access to an individual item (or set of items). In Python's case, the default start index position is 0, and for SAS it is 1.
- The slice operator `[n:m]` returns the part of the string from the n'th character to the m'th character, including the first but excluding the last. In other words, start with the character at index n and go up to but do not include the character at index m.
- If you omit the first index (before the colon), the slice starts at the beginning of the string. If you omit the second index, the slice goes to the end of the string.

Python Data Structures - String Indexing

Indexing of python strings starts from 0. For example, The string "HELLO" is indexed as shown below.

str = "HELLO"

H	E	L	L	O
0	1	2	3	4

```
[9]: #3.4 string indexing
      str = 'Hello'
      print(str[0])
      print(str[1])
      print(str[2])
      print(str[3])
      print(str[4])
```

H
e
l
l
o

Python Data Structures – String Slicing

The slice operator `[]` is used to access individual characters of the string. However, the `:` (colon) operator can be used in python to access the substring as shown below.

str = "HELLO"

H	E	L	L	O
0	1	2	3	4

```
[10]: #3.4 string slicing
str = 'HELLO'
print(str[:])
print(str[0:])
print(str[:5])
print(str[:3])
print(str[0:2])
print(str[1:4])
```

HELLO

HELLO

HELLO

HEL

HE

❏ ❏ ❏

Python Data Structures – List



- A list is an ordered set of objects in Python.
- In Python, items in a list are separated by commas and enclosed in square brackets. Lists are mutable, meaning you can add, remove, or alter items without changing their identity. Lists can be recognized by their square brackets [and] that hold elements, separated by a comma. Lists are built into Python, you do not need to invoke them separately.
- Lists are similar to arrays in SAS, C. However; Lists can contain data of different types.

Python Data Structures – Printing Index Positions

```
[12]: #3.4 List containing diff data types
c=list(['fall','10','october'])
print(type(c))
#3.4 printing index position 1
print(c[1])

#3.4 printing index position 2
print(c[2])

<class 'list'>
10
october
```

Python Data Structures - List Slicing Cheat Sheet

- Slice `[:]` operators can be used to access data in a list. The concatenation operator `(+)` and repetition operator `(*)` works with the list in the same way that they work with strings
- `a[start:stop]` # items start through stop-1
- `a[start:]` # items start through the rest of the array
- `a[:stop]` # items from the beginning through stop-1
- `a[:]` # a copy of the whole array

List Slicing

Python's list slicing lets you cut through sequences of data with precision.

```
[17]: #3.4 List slicing and operators
my = [1, 'hi', 'class', 2]
print(my[3:])
print(my[0:2])
print(my)
print(my + my)
print(my * 3)

[2]
[1, 'hi']
[1, 'hi', 'class', 2]
[1, 'hi', 'class', 2, 1, 'hi', 'class', 2]
[1, 'hi', 'class', 2, 1, 'hi', 'class', 2, 1, 'hi', 'class', 2]
```

Whether you're choosing a specific range or a particular element, it's a powerful tool to get exactly what you need.

Python List Comparison with Data Step

```
[18]: #3.4 Python code comparison with DATA step  
numbers = [2, 4, 6, 8, 10]  
product = 1  
for i in numbers:  
    product = product * i  
print('The product is:', product)
```

The product is: 3840

Python List Comparison with Data Step

```
33    /*3.4 data step comparison with Python*/  
34    data _null_;  
35        retain product 1;  
36        do i = 2 to 10 by 2;  
37            product = product * i;  
38        end;  
39        put 'The product is: ' product;  
40    run;
```

The product is: 3840

Python Basic Data Types

Boolean: Choosing Between True and False

- Boolean operations are valuable in data cleansing and transformations.
- The standard two Boolean values are True and False with the capitalization as shown. In numeric contexts, they behave like the integers with values 0 and 1, respectively.
- SAS does not have a boolean data type, so Data Step code is often constructed as a series of cascading IF-THEN/DO code blocks for testing True/False.

```
# 3.4 Booleans subclass int
```

```
active = True
```

```
admin = False
```

```
# Numeric context
```

```
n = active + admin # 1
```

```
# Test True / False
```

```
if active:
```

```
    print("On")
```


3 PROC Python : Putting it All Together

Importing Messy Lab Data

- PROC PYTHON runs Python directly inside a SAS program. Introduced in SAS Viya in 2021, it is a Viya feature today, with support planned for Base SAS 9.4 (M10) in 2027.
- Inside a SUBMIT / ENDSUBMIT block you can import open-source libraries such as pandas, NumPy, and scikit-learn.
- The built-in SAS object moves data both ways —SAS.sd2df() turns a SAS dataset into a pandas DataFrame, and SAS.df2ds() sends it back.

SAS Pandas Comparison

pandas	SAS
DataFrame	data set
column	variable
row	observation
groupby	BY-group
NaN	.

NaN is an acronym for Not a Number

Pandas – The Workhorse behind Python Wrangling



Pandas is the primary package used for working with tabular data in Python.

Its DataFrame structure is similar to a SAS data set and supports filtering, sorting, joining, summarizing, and reshaping data.

Pandas can read Excel, CSV, database, and SAS data sources. When working with Excel (.xlsx) files, pandas typically uses the openpyxl package.

Pandas Data Structures

Pandas data structure	Definition
Series	A 1-dimensional labelled array capable of holding any data type
DataFrame	A 2-dimensional labeled data structure with columns of potentially different types

Panda Components vs SAS

A DataFrame looks a great deal like a SAS data set (or relational table). The table below compares panda components to those found in SAS.

Pandas	SAS
DataFrame	SAS data set
row	observation
column	variable
groupby	BY-Group
NaN	.
slice	sub-set
axis 0	observation
axis 1	column

Importing From LabExtract.xlsx

Columns Exactly as In Import File

The CONTENTS Procedure

Data Set Name	WORK.DIRTY	Observations	3
Member Type	DATA	Variables	20
Engine	V9	Indexes	0
Created	07/16/2026 11:33:22	Observation Length	160
Last Modified	07/16/2026 11:33:22	Deleted Observations	0
Protection		Compressed	NO
Data Set Type		Sorted	NO
Label			
Data Representation	SOLARIS_X86_64, LINUX_X86_64, ALPHA_TRU64, LINUX_IA64, LINUX_POWER_64		

Variables in Creation Order					
#	Variable	Type	Len	Format	Label
1	ID	Num	8	BEST.	ID
2	Hematology Measurement;Mean;Meas	Num	8	BEST.	Hematology Measurement;Mean;Measurement;Meas. Type: White Blood Cell;Units: E9/L
3	Hematology Measurement;Mean;Me_1	Num	8	BEST.	Hematology Measurement;Mean;Measurement;Meas. Type: Neutrophil;Units: E9/L
4	Hematology Measurement;Mean;Me_2	Num	8	BEST.	Hematology Measurement;Mean;Measurement;Meas. Type: Lymphocytes;Units: E9/L
5	Hematology Measurement;Mean;Me_3	Num	8	BEST.	Hematology Measurement;Mean;Measurement;Meas. Type: Monocytes;Units: E9/L
6	Hematology Measurement;Mean;Me_4	Num	8	BEST.	Hematology Measurement;Mean;Measurement;Meas. Type: Eosinophil;Units: E9/L
7	Hematology Measurement;Mean;Me_5	Num	8	BEST.	Hematology Measurement;Mean;Measurement;Meas. Type: Basophil;Units: E9/L
8	Hematology Measurement;Mean;Me_6	Num	8	BEST.	Hematology Measurement;Mean;Measurement;Meas. Type: Red Blood Cells;Units: E12/L
9	Hematology Measurement;Mean;Me_7	Num	8	BEST.	Hematology Measurement;Mean;Measurement;Meas. Type: Hemoglobin;Units: g/dL

Summon the Python

1 Read Excel

SAS: start a Python procedure

```
proc python;  
submit;
```

Load pandas, the library for working with tables/DataFrames

```
import pandas as pd
```

Define a reusable function: takes an input path and an output path

```
def update_column_names(input_file, output_file):
```

Read the Excel file using the openpyxl engine

```
df = pd.read_excel(input_file, engine="openpyxl")
```


Summon the Python

2 Loop through column names

```
new_columns = []  
    for column in df.columns:
```

Summon the Python

3 Extract cleaner names

```
index_of_colon = column.find(":")  
index_of_semicolon = column.find(";", index_of_colon)  
new_column_name = column[index_of_colon + 1:].strip()  
  
new_columns.append(new_column_name)
```

Summon the Python

4 Write the clean workbook back to Excel

```
df.columns = new_columns
```

```
df.to_excel(output_file, index=False)
```

Success!

Columns after Python Code Execution
The CONTENTS Procedure

Data Set Name	WORK.IMPORT	Observations	3
Member Type	DATA	Variables	20
Engine	V9	Indexes	0
Created	04/18/2025 11:20:18	Observation Length	160
Last Modified	04/18/2025 11:20:18	Deleted Observations	0
Protection		Compressed	NO
Data Set Type		Sorted	NO
Label			
Data Representation	SOLARIS_X86_64, LINUX_X86_64, ALPHA_TRU64, LINUX_IA64, LINUX_POWER_64		
Encoding	utf-8 Unicode (UTF-8)		

Variables in Creation Order					
#	Variable	Type	Len	Format	Label
1	ID	Num	8	BEST.	ID
2	White_Blood_Cell	Num	8	BEST.	White Blood Cell
3	Neutrophil	Num	8	BEST.	Neutrophil
4	Lymphocytes	Num	8	BEST.	Lymphocytes
5	Monocytes	Num	8	BEST.	Monocytes
6	Eosinophil	Num	8	BEST.	Eosinophil
7	Basophil	Num	8	BEST.	Basophil
8	Red_Blood_Cells	Num	8	BEST.	Red Blood Cells

Handy Links

- [Github link for all files used in the presentation and demo](#)
- [Comparison with SAS \(pandas Documentation\)](#) – Maps SAS DATA step and PROC operations to their pandas equivalents.
- [Getting Started with Python Programming on SAS Viya \(SAS Help Center\)](#) – Official guide to running Python and PROC PYTHON inside a SAS program.
- [SASPy Documentation](#) – Interface for running SAS from Python and moving data between the two.
- [pandas Documentation](#) – Reference for the DataFrame library used for data wrangling in Python.
- [NumPy Documentation](#) – Core numeric computing package that underpins pandas and analytics workflows.
- [SAS Viya Copilot](#) – AI coding assistant that suggests, explains, and validates Python and SAS code.

Thank You

Charu Shankar
SAS Institute Toronto

EMAIL Charu.shankar@sas.com

BLOG <https://blogs.sas.com/content/author/charushankar/>

X (TWITTER) CharuYogaCan

LINKEDIN <https://www.linkedin.com/in/charushankar/>

✓ Did you
enjoy this
session, Let us
know in the
[evaluation](#)

